

Minuta de Clase — Aspectos Avanzados de Programación Funcional

Guion docente — Tema 04

Matías Gel

Ciclo lectivo 2026

Índice

Clase: Aspectos Avanzados de Programación Funcional	1
Objetivos de la Clase	1
BLOQUE 1 — Fundamentos avanzados (35 min)	2
BLOQUE 2 — Abstracciones y efectos (35 min)	10
BLOQUE 3 — Concurrencia y metaprogramación (30 min)	21
BLOQUE 4 — Práctica guiada y reflexión (20 min)	30
Materiales y recursos en clase	34
Trazabilidad a filminas	34

Clase: Aspectos Avanzados de Programación Funcional

Materia: Paradigmas y Lenguajes de Programación 2026 — UNTDF / IDEI **Tema:** 04 | **Módulo:**

II Duración: 120 minutos | **Fecha:** _____

Cómo usar esta minuta: Cada sección corresponde a una filmina. Los momentos () son acciones secuenciales dentro de esa filmina. El texto entrecomillado es lo que decís en voz alta. El código está inline — no necesitás abrir `filminas.md` para dar la clase.

Objetivos de la Clase

- OA-1: Comparar la expresión de patrones funcionales en TypeScript y Clojure.
- OA-2: Aplicar transformaciones de datos con funciones puras y colecciones inmutables.
- OA-3: Construir y usar `Result` / `Either` para manejo funcional de errores.
- OA-4: Entender transducers en Clojure y su equivalente en TS como composición de transformaciones.
- OA-5: Identificar modelos de concurrencia funcional en Clojure y contrastarlos con `Promise` / `async-await` en TypeScript.
- OA-6: Diseñar una pequeña API funcional generics-safe en TypeScript.

BLOQUE 1 — Fundamentos avanzados (35 min)

[F-01] Portada

Tiempo: 1 min

Al mostrar la portada > “Hoy vamos a avanzar más allá de los fundamentos funcionales. Vamos a comparar cómo se resuelven los mismos problemas en TypeScript y Clojure, y cómo ese contraste nos ayuda a elegir buenas abstracciones.”

Transición: “Comenzamos con una regla simple: menos estado, más composición.”

[F-02] ¿Por qué hablar de funcional?

Tiempo: 3 min

Al mostrar la regulación > “El funcional no es un capricho académico. Es la respuesta a problemas reales de concurrencia, escala y mantenibilidad.”

Enfatizar - El estado mutable complica el desarrollo concurrente. - El funcional reduce los efectos secundarios. - Las ideas que veremos hoy aplican en TS y Clojure.

Transición: “Veamos cómo se comparan los paradigmas.”

[F-03] Imperativo vs funcional

Tiempo: 3 min

Al mostrar la tabla > “En el paradigma imperativo describimos *cómo* llegar al resultado paso a paso: variables que mutan, bucles que incrementan contadores, instrucciones que se ejecutan en orden. En el funcional describimos *qué* es el resultado: composición de funciones sin estado observable entre ellas.”

Conceptos clave para desarrollar: - **Imperativo:** estado mutable, secuencia de instrucciones, efectos secundarios en cualquier punto. Ejemplo mental: un `for` que acumula en una variable externa. - **Funcional:** expresiones en lugar de sentencias. Cada función recibe un valor y devuelve uno nuevo sin modificar nada externo. El flujo de control es composición. - Ambos son Turing-completos — la diferencia está en el *modelo mental* y en *dónde viven los errores*. El imperativo concentra complejidad en el estado; el funcional la concentra en los tipos. - Un mismo problema puede resolverse en ambos paradigmas: la elección afecta la capacidad de razonar sobre el código, no su potencia computacional.

“Cuando necesitamos controlar estado muy fino — un motor de juego con objetos que colisionan, por ejemplo — el imperativo es natural y eficiente. Cuando trabajamos con transformaciones de datos, paralelismo o flujos de eventos, el funcional nos da una ventaja enorme en predecibilidad.”

Pregunta: “¿Qué paradigma prefieren cuando necesitan controlar estado muy fino? ¿Y cuándo procesan grandes volúmenes de datos?”

Transición: “Ahora definamos con más precisión qué es una función pura.”

[F-04] Funciones puras

Tiempo: 3 min

Al mostrar la definición > “Una función pura no mira ni modifica nada que no esté en sus parámetros. Eso nos da predictibilidad y facilita pruebas.”

Ejemplos: - `const doble = x => x * 2` - `const area = ({ radius }) => Math.PI * radius ** 2`

Transición: “Ahora conectemos esto con la inmutabilidad.”

[F-05] Inmutabilidad

Tiempo: 3 min

Al mostrar la comparación > “En un programa imperativo, un objeto puede ser modificado desde cualquier lugar que tenga una referencia. Eso crea bugs difíciles de rastrear: alguien mutó el arreglo y no sabemos quién, cuándo ni por qué.”

Conceptos clave para desarrollar: - **Mutabilidad:** `let x = 5; x = 10;` — el binding cambia. En arrays: `arr.push(6)` modifica el array original. Cualquier función que reciba esa referencia ve el cambio. - **Inmutabilidad:** en lugar de mutar, creamos un valor nuevo. `const nuevo = [...arr, 6]` — `arr` sigue intacto. La función original no sabe que existió una versión extendida. - El costo aparente es memoria extra, pero las estructuras de datos persistentes implementadas en lenguajes como Clojure comparten nodos internos (structural sharing): el costo real es logarítmico, no lineal. - El beneficio clave: **ningún thread necesita un lock para leer un valor que sabe que nadie va a cambiar**. La sincronización costosa desaparece.

Código de contraste para la pizarra:

```
// Mutable - peligroso en concurrencia
const inventario = ["A", "B"];
funcionExterna(inventario); // ¿qué hizo con inventario?
console.log(inventario); // no sabemos

// Immutable - predecible
const inventario = ["A", "B"] as const;
const actualizado = [...inventario, "C"]; // inventario intacto
```

Punto clave: > “La inmutabilidad es la mejor defensa contra bugs de concurrencia. No por magia, sino porque elimina la categoría de error: si nadie puede mutar el valor, el bug de estado compartido simplemente no puede existir.”

Transición: “Veamos cómo se ve esto en TypeScript.”

[F-06] Pipeline en TypeScript

Tiempo: 3 min

Al mostrar el código > “Un pipeline funcional es una secuencia de transformaciones donde la salida de cada función es la entrada de la siguiente. Ninguna modifica el array original.”

Código inline del pipeline:

```
const ordenes = [
  { id: 1, total: 120, activa: true },
  { id: 2, total: 50, activa: false },
  { id: 3, total: 200, activa: true },
];

const totalActivas = ordenes
  .filter(o => o.activa)           // nuevo array: [{ id:1 }, { id:3 }]
  .map(o => o.total)              // nuevo array: [120, 200]
  .reduce((acc, t) => acc + t, 0); // valor: 320

console.log(ordenes.length); // 3 - intacto
```

Recorrido paso a paso: 1. `.filter(o => o.activa)` — evalúa el predicado para cada elemento y devuelve un *nuevo* array con los que pasaron. No muta `ordenes`. 2. `.map(o => o.total)` — transforma cada objeto en su campo `total`. Devuelve un *nuevo* array de números. 3. `.reduce((acc,`

`t) => acc + t, 0)` — acumula todos los valores comenzando desde 0. Devuelve un escalar.

“Cuando encadenamos estos métodos, JavaScript no evalúa nada hasta que se llama al método final. Cada uno recibe el resultado del anterior como si fuera su propio array de entrada.”

Pregunta: “¿Qué valor devuelve el método `filter`? ¿Qué tipo tiene?”

Transición: “Ahora desglosamos cada operación.”

[F-07] `filter` + `map`

Tiempo: 3 min

Al mostrar la tabla > “`filter` y `map` son las dos operaciones fundamentales de todo pipeline funcional. Entenderlas bien es entender el 80% del paradigma.”

`filter` — selección con predicado:

```
// Firma conceptual: filter<T>(pred: (x: T) => boolean): T[]
const pares = [1, 2, 3, 4].filter(x => x % 2 === 0); // [2, 4]
```

- Recibe una función booleana (predicado). Incluye el elemento si el predicado devuelve `true`.
- El array original no cambia. El resultado tiene igual o menor cantidad de elementos.

`map` — transformación elemento a elemento:

```
// Firma conceptual: map<T, U>(fn: (x: T) => U): U[]
const cuadrados = [2, 4].map(x => x * x); // [4, 16]
```

- Recibe una función de transformación. El resultado siempre tiene la *misma cantidad* de elementos.
- El tipo puede cambiar: podemos ir de `number[]` a `string[]`.

Relación con composición: > “Componer `filter` con `map` es la combinación más usada en programación funcional. Primero reducimos el conjunto, luego transformamos. En equipo: $[1,2,3,4] \rightarrow \text{filter}(\text{par}) \rightarrow [2,4] \rightarrow \text{map}(x^2) \rightarrow [4,16]$.”

Ejemplo rápido: - De $[1,2,3,4] \rightarrow \text{filter}(x \% 2 === 0) \rightarrow [2,4] \rightarrow \text{map}(x * x) \rightarrow [4,16]$

Transición: “Seguimos con `reduce`.”

[F-08] reduce

Tiempo: 3 min

Al mostrar el código > “reduce es la operación más poderosa y versátil del paradigma funcional. Cualquier acumulación, agrupación o síntesis de una colección puede expresarse con **reduce**.”

Firma conceptual:

```
// reduce<T, U>(fn: (acc: U, cur: T) => U, initialValue: U): U
```

Ejemplo de suma:

```
const nums = [1, 2, 3, 4, 5];
const suma = nums.reduce((acc, x) => acc + x, 0);
// paso 1: acc=0, x=1 → 1
// paso 2: acc=1, x=2 → 3
// paso 3: acc=3, x=3 → 6
// paso 4: acc=6, x=4 → 10
// paso 5: acc=10, x=5 → 15
```

Contrastar con el loop imperativo equivalente:

```
// Imperativo:
let suma = 0;
for (const x of nums) { suma += x; } // muta `suma`

// Funcional: el acumulador `acc` nunca muta - es un parámetro nuevo en cada llamada
```

“La diferencia clave: en el **for** imperativo, **suma** es una variable que se modifica. En **reduce**, **acc** es un *parámetro* nuevo en cada invocación de la función. No hay mutación — solo pasaje de valores.”

Casos de uso avanzados (mencionar brevemente): - Construir un objeto desde un array: `reduce((acc, item) => ({ ...acc, [item.id]: item }), {})` - Implementar **map** y **filter** con **reduce** (ejercicio mental potente)

Transición: “Hagamos el pipeline más reusable con composición.”

[F-09] Composición en TypeScript

Tiempo: 3 min

Al mostrar el patrón > “En lugar de encadenar métodos, podemos construir funciones

reutilizables que describen transformaciones independientemente de los datos. Eso es composición.”

pipe — iz a der (más intuitivo):

```
const pipe = (...fns: Function[]) => (x: any) =>
  fns.reduce((v, f) => f(v), x);

const soloActivas = (ordenes: Orden[]) => ordenes.filter(o => o.activa);
const extraerTotal = (ordenes: Orden[]) => ordenes.map(o => o.total);
const sumar = (nums: number[]) => nums.reduce((a, b) => a + b, 0);

const totalActivas = pipe(soloActivas, extraerTotal, sumar);
console.log(totalActivas(ordenes)); // 320
```

compose — der a izq (orden matemático):

```
const compose = (...fns: Function[]) => (x: any) =>
  fns.reduceRight((v, f) => f(v), x);
// compose(f, g, h)(x) === f(g(h(x)))
```

¿Por qué importa? > “La función `totalActivas` que construimos con `pipe` es completamente reutilizable. No está atada a ningún array particular. Podemos probarla en aislamiento, componerla con otras funciones y nombrarla con semántica del dominio.”

Conexión con transducers (adelanto): - `pipe` crea colecciones intermedias en cada paso. Los transducers eliminan ese costo. Lo veremos en F-19.

Transición: “Pasamos a Clojure para ver el mismo patrón.”

[F-10] Colecciones inmutables en TS

Tiempo: 3 min

Al mostrar ejemplos > “TypeScript no tiene inmutabilidad profunda incorporada, pero tenemos herramientas para acercarnos.”

as const — congelar un literal:

```
const persona = { name: "Ana", role: "admin" } as const;
// persona.name = "Juan"; // Error de compilación
// El tipo inferido es: { readonly name: "Ana"; readonly role: "admin" }
```

Spread — actualización inmutable de objetos:

```
const persona = { name: "Ana", age: 28 };
const cumpleaños = { ...persona, age: 29 }; // nuevo objeto
console.log(persona.age); // 28 - intacto
console.log(cumpleaños.age); // 29
```

Arrays inmutables con spread:

```
const lista = [1, 2, 3];
const conCuatro = [...lista, 4]; // nuevo array - lista sigue siendo [1,2,3]
const sinUno = lista.slice(1); // [2, 3] - lista intacta
```

readonly en interfaces:

```
interface Punto { readonly x: number; readonly y: number; }
const mover = (p: Punto, dx: number): Punto => ({ x: p.x + dx, y: p.y });
// No podemos mutar p adentro de la función aunque intentemos
```

Punto clave: > “La versión original sigue intacta. Cada transformación produce un *nuevo* valor. Esto hace que las funciones sean predecibles y testeables en aislamiento.”

Transición: “En Clojure esto no es una convención ni requiere disciplina — es la única forma de trabajar.”

[F-11] Secuencias perezosas en Clojure

Tiempo: 3 min

Al explicar > “Clojure evalúa colecciones de forma perezosa (*lazy*): define qué se va a hacer, pero no lo hace hasta que alguien pide el valor. Esto cambia profundamente cómo pensamos las transformaciones.”

Ejemplo en Clojure:

```
;; map devuelve un lazy-seq, no evalúa aún
(def dobles (map #(* 2 %) (range 1 1000000)))

;; Recién al pedir los primeros 5, se evalúan esos 5:
(take 5 dobles) ; => (2 4 6 8 10)
```

¿Por qué es una ventaja? - Trabajar con colecciones infinitas es posible: `(range)` produce todos los enteros, `(take 10 (range))` toma solo 10. - No se crean colecciones intermedias grandes si solo necesitamos parte del resultado. - Las transformaciones se fusionan: `(map f (filter g coll))` procesa cada elemento una sola vez.

Contraste con TypeScript: > “En TypeScript, `[1,2,3].filter(...).map(...)` crea DOS arrays intermedios completos. En Clojure, las secuencias perezosas modelan el pipeline sin materializar cada paso. Los transducers en Clojure llevan esto aún más lejos — lo veremos pronto.”

Transición: “Veamos el pipeline de Clojure completo.”

[F-12] Pipeline en Clojure

Tiempo: 3 min

Al mostrar el código > “En Clojure el pipeline se escribe con el macro `->>`, que toma un valor inicial y lo pasa como último argumento a cada función encadenada. Es el equivalente funcional de los métodos encadenados de TypeScript — pero más general: funciona con *cualquier* función, no solo métodos de una clase.”

Código inline:

```
(def ordenes
  [{:id 1 :total 120 :activa? true}
   {:id 2 :total 50  :activa? false}
   {:id 3 :total 200 :activa? true}])

(->> ordenes
  (filter :activa?)      ; [{:id 1 ...} {:id 3 ...}]
  (map :total)           ; (120 200)
  (reduce + 0))         ; => 320
```

Cómo leer `->>` en voz alta: > “Tomá `ordenes`, filtrá las activas, extraé el total de cada una, y sumá todo.”

Puntos para desarrollar: - `(filter :activa? ordenes)` — en Clojure, un keyword como `:activa?` es una función que busca su valor en un mapa. Economía de código. - `(map :total ...)` — lo mismo: `:total` como función extrae el campo de cada mapa. - `->>` simplemente reordena los argumentos para que el flujo de datos sea legible de arriba a abajo. - Comparar con `->` (threading macro normal, pasa como primer argumento) vs `->>` (pasa como último).

Transición: “Y ahora, la razón por la que las colecciones en Clojure son tan eficientes a pesar de ser inmutables.”

[F-13] Colecciones persistentes en Clojure

Tiempo: 3 min

Al explicar > “Cuando oímos ‘inmutabilidad’, nos preocupa el costo: si cada operación crea un nuevo valor, ¿no vamos a consumir memoria? En Clojure, la respuesta es no — por algo llamado *structural sharing* (compartición de estructura).”

Cómo funciona structural sharing:

```
(def v1 [1 2 3 4 5])  
(def v2 (conj v1 6)) ; "nuevo" vector con el 6 añadido  
  
;; v2 NO hace una copia completa de v1.  
;; v2 COMPARTE los nodos internos de v1.  
;; Solo se crea la diferencia (el nodo nuevo + un puntero).
```

Analogía para explicar en clase: > “Imaginen un árbol genealógico. Cuando nace un hijo, no reescribimos toda la familia — solo agregamos un nodo nuevo que apunta al padre existente. Las colecciones persistentes de Clojure funcionan igual: son árboles donde las versiones comparten ramas.”

Implementación interna: - Los vectores y mapas de Clojure son Hash Array Mapped Tries (HAMTs) — árboles balanceados de fanout 32. - Operación de `conj` o `assoc`: $O(\log n)$ $O(1)$ en la práctica. - La GC de la JVM recoge los nodos que ninguna versión referencia.

Punto clave: > “La inmutabilidad en Clojure es eficiente por diseño. No es un compromiso entre corrección y performance — los diseñadores resolvieron ese problema con estructuras de datos persistentes. Podemos tener ambas cosas.”

BLOQUE 2 — Abstracciones y efectos (35 min)

[F-14] Algebraic data types en TS

Tiempo: 3 min

Al mostrar la definición > “Un tipo algebraico es un tipo que se forma *combinando* otros tipos. El nombre viene del álgebra: podemos combinar tipos como sumamos o multiplicamos. Son la base de cómo el sistema de tipos describe todos los resultados posibles de una operación.”

Dos clases de tipos algebraicos:

1. Tipo producto (AND): combina varios campos simultáneamente. Si A tiene m valores y B tiene n valores, $A \times B$ tiene $m \times n$ valores posibles.

```
// Un par de valores - los dos deben estar presentes a la vez
type Punto = { x: number; y: number };
// Un Punto "tiene" x AND y
```

2. Tipo suma (OR / unión discriminada): es *uno* de varios posibles casos. Los valores totales son la suma de cada variante.

```
// Un resultado es OK o es Error - nunca los dos a la vez
type Result<T, E> =
  | { ok: true; value: T } // variante "éxito"
  | { ok: false; error: E }; // variante "fallo"
```

“Los tipos suma son la clave. En lugar de tener una función que puede devolver un número *o* lanzar una excepción *o* devolver null — el tipo algebraico describe *todas* esas posibilidades en la firma. El compilador nos obliga a manejarlas.”

Ejemplo más completo:

```
type Shape =
  | { kind: "circle"; radius: number }
  | { kind: "rectangle"; width: number; height: number }
  | { kind: "triangle"; base: number; height: number };

// El compilador sabe que si kind === "circle",
// el campo radius existe. No necesitamos castings.
const area = (s: Shape): number => {
  switch (s.kind) {
    case "circle": return Math.PI * s.radius ** 2;
    case "rectangle": return s.width * s.height;
    case "triangle": return 0.5 * s.base * s.height;
  }
};
```

Transición: “El caso de uso más poderoso de los tipos suma es el manejo de errores. Veamos cómo `Result` reemplaza a las excepciones.”

[F-15] **Result vs excepción**

Tiempo: 3 min

Al mostrar la tabla > “Cuando una operación puede fallar — dividir por cero, leer un archivo

que no existe, parsear JSON malformado — la respuesta tradicional es lanzar una excepción. El problema: las excepciones son *invisibles* en la firma de la función.”

Tabla de comparación para desarrollar:

Dimensión	Excepción (<code>throw</code>)	<code>Result<T, E></code>
Firma de la función	<code>divide(a, b): number</code> — el error es invisible	<code>divide(a, b): Result<number, string></code> — el error es parte del tipo
Propagación	Se propaga silenciosamente por la pila	Debe manejarse explícitamente antes de continuar
Composición	Dificulta el encadenamiento (<code>try/catch</code> anidados)	Se puede encadenar con <code>map</code> , <code>flatMap</code>
Rendimiento	Costoso: construye stack trace	Sin overhead: es solo una unión de tipos

“Con `Result`, el que llama a la función *sabe* que puede fallar. No puede ignorarlo: el tipo lo obliga a bifurcar. Las excepciones no capturadas son la principal fuente de crashes en producción.”

Pregunta: “¿Qué ocurre si no manejamos una excepción? ¿Dónde termina en una aplicación real?”

[F-16] Ejemplo `Result` en TS

Tiempo: 3 min

Al mostrar el código > “Definimos el tipo `Result` y una función que lo usa. Recorramos cada rama.”

Definición del tipo y helpers:

```
type Result<T, E = string> =
  | { ok: true; value: T }
  | { ok: false; error: E };

const ok = <T>(value: T): Result<T, never> => ({ ok: true, value });
const err = <E>(error: E): Result<never, E> => ({ ok: false, error });
```

Función que devuelve `Result`:

```
const dividir = (a: number, b: number): Result<number, string> =>
  b === 0
    ? err("División por cero")
    : ok(a / b);
```

Consumo con pattern matching explícito:

```
const resultado = dividir(10, 0);

if (resultado.ok) {
  console.log("Resultado:", resultado.value); // TypeScript sabe que .value existe
} else {
  console.error("Error:", resultado.error); // y que .error existe en el else
}
```

Encadenamiento (programación defensiva):

```
const procesarDivision = (a: number, b: number): Result<string, string> => {
  const r = dividir(a, b);
  if (!r.ok) return r; // propagar el error
  const redondo = Math.round(r.value);
  return ok(`El resultado redondeado es ${redondo}`);
};
```

“Cada paso que puede fallar devuelve un Result. Si algo falla, propagamos el error hacia arriba. El camino feliz solo se ejecuta cuando todo estuvo bien. No hay excepciones que se escapen silenciosamente.”

Transición: “Ahora un patrón equivalente en Clojure.”

[F-17] Option / Maybe

Tiempo: 3 min

Al explicar > “Maybe (o Option) es un tipo algebraico más simple que Result: modela la posibilidad de *ausencia* de un valor, sin agregar información de error.”

Definición:

```
type Maybe<T> =
  | { some: true; value: T }
  | { some: false };
```

```
const just    = <T>(value: T): Maybe<T> => ({ some: true, value });
const nothing = <T>(): Maybe<T>        => ({ some: false });
```

Comparación con null / undefined:

```
// Con null - puede fallar silenciosamente:
const buscarUsuario = (id: number): Usuario | null => ...;
const u = buscarUsuario(99);
console.log(u.nombre); // TypeError si u es null - NO detectado en compilación

// Con Maybe - el compilador nos obliga a verificar:
const buscarUsuario = (id: number): Maybe<Usuario> => ...;
const u = buscarUsuario(99);
if (u.some) {
  console.log(u.value.nombre); // seguro
}
```

Diferencia conceptual con Result: | Maybe<T> | Result<T, E> | —|—|—| | Falla | Sí / No |
 Sí, con razón tipada | Información de error | No tiene | El tipo E describe el fallo | Uso típico |
 Búsquedas en colección, campos opcionales | Operaciones que pueden fallar con contexto |

“La regla es simple: si la ausencia de valor es suficiente información, usamos **Maybe**. Si el consumidor necesita saber *por qué* falló, usamos **Result**. Modelar la ausencia con tipos en lugar de null elimina una clase entera de runtime errors.”

Punto clave: > “Es mejor modelar la ausencia de valor con tipos, no con valores especiales como null o -1. El sistema de tipos se convierte en documentación ejecutable.”

[F-18] Manejo de errores en Clojure

Tiempo: 4 min

Al mostrar el código > “Clojure no tiene el sistema de tipos estático de TypeScript, pero tiene la misma filosofía: convertir errores en *datos* en lugar de en *excepciones*.”

Función dividir en Clojure:

```
(defn dividir [a b]
  (if (zero? b)
    {:ok false :error "División por cero"}
    {:ok true  :value (/ a b)}))
```

```
;; Uso:
(let [resultado (dividir 10 0)]
  (if (:ok resultado)
      (println "Resultado:" (:value resultado))
      (println "Error:"      (:error resultado)))))
```

Recorrer la función paso a paso: 1. (zero? b) — predicado que verifica si b es cero. En Clojure, las funciones de predicado terminan en ? por convención. 2. Si es cero, devuelve un **mapa** con clave :ok false y un mensaje de error. 3. Si no, devuelve un mapa con :ok true y el valor calculado. 4. El resultado es **siempre un mapa** — no hay excepción posible. El llamador maneja ambos casos.

¿Por qué esto es poderoso en Clojure? > “En Clojure, los datos son ciudadanos de primera clase. Los mapas son la estructura universal. Modelar el error como un mapa nos da gratis: podemos logearlo, serializarlo a JSON, agregarlo a una cola de eventos, compararlo en tests. Las excepciones son opacas; los datos son transparentes.”

Discusión breve: > “Este patrón convierte errores en datos. ¿Alguien puede pensar en un caso donde esto sea una desventaja frente a las excepciones? (Respuesta esperada: cuando el error es verdaderamente excepcional y no hay nada que hacer — como corrupción de memoria. Ahí las excepciones tienen sentido.)”

[F-19] ¿Qué es un transducer?

Tiempo: 3 min

Al mostrar la definición > “Un transducer es una transformación composable e independiente de la colección sobre la que opera. La palabra viene de *transformer reducer* — es una función que transforma un reductor para crear otro.”

El problema que resuelven (motivación):

```
// Pipeline clásico - crea 2 arrays intermedios:
array
  .filter(esPar)    // Array intermedio 1
  .map(duplicar);   // Array intermedio 2
```

- Si el array tiene 1 millón de elementos, creamos un millón extra en cada paso.
- No podemos reutilizar ese pipeline para procesar un **Set**, un **Stream** o una base de datos.

La idea clave: > “Un transducer describe la transformación de forma abstracta, sin mencionar la fuente ni el destino de los datos. Es como una ‘receta de procesamiento’ que puede aplicarse a

cualquier fuente.”

En Clojure — cómo se construye:

```
;; filter y map retornan transducers cuando se llaman sin colección:
(def xf
  (comp
    (filter even?)    ; transducer: filtrar pares
    (map #(* 2 %))    ; transducer: duplicar
  ))

;; xf es una transformación que aún no tiene datos
```

Échale datos:

```
(transduce xf + [1 2 3 4 5 6])
;; Solo los pares: 2 4 6
;; Duplicados: 4 8 12
;; Reducción con +: 24
;; Sin colección intermedia
```

Relación con compose en TS: - El equivalente conceptual en TS sería una versión de pipe que fusiona las operaciones para eliminar intermedios. Librerías como `transducers-js` lo implementan.

Transición: “Veamos un ejemplo completo de transduce.”

[F-20] Ejemplo de transduce

Tiempo: 3 min

Al mostrar el código > “Ahora veamos transduce con un ejemplo de dominio: procesar órdenes de compra.”

Código completo:

```
(def ordenes
  [{:id 1 :total 300 :activa? true}
   {:id 2 :total 80 :activa? false}
   {:id 3 :total 150 :activa? true}
   {:id 4 :total 500 :activa? true}])

;; Definir el transducer: activas, mayor a 100, extraer total
(def xf
```



```

(comp
  (filter :activa?)
  (filter #(> (:total %) 100))
  (map :total)))

;; Aplicar a la colección, reducir con suma:
(transduce xf + ordenes)
;; => 950 (300 + 150 + 500)

```

Recorrido paso a paso: 1. `(filter :activa?)` — transducer que pasa solo los activos. 2. `(filter #(> (:total %) 100))` — transducer que pasa solo los de `total > 100`. 3. `(map :total)` — transducer que extrae el campo `:total`. 4. `comp` los combina de derecha a izquierda (matemáticamente), pero el flujo de datos de izquierda a derecha. 5. `transduce xf + ordenes` — aplica la transformación y reduce con `+`, sin crear colecciones intermedias.

“Los tres pasos se fusionan en un solo recorrido del array. Cada orden se evalúa contra todos los filtros y se mapea en el mismo pase. No hay estructuras intermedias.”

Transición: “Veamos por qué eso importa en benchmarks reales.”

[F-21] Transducers vs pipeline convencional

Tiempo: 3 min

Al mostrar la tabla > “Compararemos el pipeline clásico con transducer en tres ejes: memoria, reutilización y composición.”

Dimensión	Pipeline clásico	Transducer
Colecciones intermedias	Una por cada <code>filter/map</code>	Ninguna
Recorridos del array	N (uno por operación)	1 (un único recorrido fusionado)
Reutilización	La cadena está ligada al tipo de colección	Aplica a arrays, streams, canales, etc.
Early termination	Imposible en el medio del pipeline	Posible con <code>(take n)</code>
Legibilidad	Muy alta para casos simples	Algo más abstracta, pero potente

Cuándo usar cada uno: > “Para datasets pequeños o medianos, el pipeline clásico es más legible y

más que suficiente. Los transducers brillan cuando procesamos colecciones grandes, manejaos flujos de datos continuos (como eventos en tiempo real) o necesitamos la misma lógica sobre varias fuentes distintas.”

Ejemplo de reutilización entre fuentes:

```
;; Mismo transducer, distintas fuentes:  
(transduce xf + lista-de-ordenes)           ; desde un vector  
(transduce xf + (sequence xf stream)) ; desde un stream de eventos  
(into [] xf lista-de-ordenes)             ; materializar a vector
```

[F-22] API funcional genérica en TS

Tiempo: 3 min

Al mostrar el código > “Los genéricos en TypeScript nos permiten escribir funciones que trabajan con *cualquier* tipo, manteniendo la información de tipo a lo largo del pipeline. Eso es una API funcional genuinamente reusable y type-safe.”

Ejemplo de API genérica:

```
// Función genérica que mapea sobre Result:  
const mapResult = <T, U, E>(  
  result: Result<T, E>,  
  fn: (value: T) => U  
) : Result<U, E> =>  
  result.ok ? ok(fn(result.value)) : result;  
  
// Funciona con cualquier tipo T y U:  
const r1 = ok(10);  
const r2 = mapResult(r1, x => x * 2); // Result<number, never>  
const r3 = mapResult(r1, x => `${x}`) // Result<string, never>
```

flatMap para encadenar operaciones que pueden fallar:

```
const flatMapResult = <T, U, E>(  
  result: Result<T, E>,  
  fn: (value: T) => Result<U, E>  
) : Result<U, E> =>  
  result.ok ? fn(result.value) : result;
```

```
// Encadenar dos operaciones que pueden fallar:
const parsear = (s: string): Result<number, string> =>
  isNaN(+s) ? err("No es un número") : ok(+s);

const proceso = flatMapResult(parsear("42"), dividir10);
```

¿Por qué esto es una API? > “Las funciones `mapResult` y `flatMapResult` son contratos genéricos que cualquier código puede usar. Al mantener los tipos paramétricos, el compilador verifica que los tipos fluyan correctamente por toda la cadena. Eso es lo que hace una API funcional *generics-safe*.”

Conexión con mónadas (adelanto): - Lo que describimos — un tipo contenedor con `map` y `flatMap` — es una mónada. El siguiente tema profundiza en eso.

[F-23] Funciones de orden superior

Tiempo: 3 min

Al explicar > “Una función de orden superior es una función que toma una o más funciones como argumentos, o devuelve una función como resultado. Son el mecanismo que hace posible toda la composición que vimos hoy.”

Casos de orden superior:

```
// 1. Toma una función como argumento:
const aplicarDosVeces = <T>(f: (x: T) => T) => (x: T): T => f(f(x));
const duplicar = (x: number) => x * 2;
console.log(aplicarDosVeces(duplicar)(3)); // 12

// 2. Devuelve una función (currying / clausura):
const multiplicarPor = (factor: number) => (x: number) => x * factor;
const triple = multiplicarPor(3); // nueva función
console.log(triple(7)); // 21

// 3. `map`, `filter`, `reduce` son de orden superior:
[1,2,3].map(x => x + 1); // map recibe la función (x => x + 1)
```

En Clojure — las funciones son valores de primera clase:

```
;; Devolver una función:
(defn multiplicar-por [factor]
```

```

(fn [x] (* factor x)))

(def triple (multiplicar-por 3))
(triple 7) ; => 21

;; Tomar una función:
(defn aplicar-dos-veces [f x] (f (f x)))
(aplicar-dos-veces inc 5) ; => 7

```

“Cada vez que pasamos `o => o.activa` a `.filter()`, escribimos una función anónima de orden superior. Cada transducer que construimos con `comp` *devuelve* una función. Toda la composición funcional descansa en este mecanismo.”

Punto clave: > “Las funciones de orden superior son el corazón de la composición. Sin ellas, no habría `map`, `filter`, `compose` ni transducers. Son el mecanismo de extensión del paradigma funcional.”

[F-24] Metaprogramación en Clojure

Tiempo: 4 min

Al explicar > “Clojure es un Lisp. Los Lisps tienen una característica única en el mundo de los lenguajes de alto nivel: la metaprogramación con macros. Un macro es código que se ejecuta en tiempo de compilación y produce código.”

¿Qué lo hace especial? - En Clojure, el código es datos (listas). Un macro recibe una lista de código sin evaluar y devuelve una lista nueva — que es el código que efectivamente se compilará. - Esto se llama *homoiconicidad*: el lenguaje de datos y el lenguaje de código son el mismo.

El macro `->>` que usamos antes:

```

;; Esto que escribimos:
(->> ordenes
  (filter :activa?)
  (map :total)
  (reduce + 0))

;; Es expandido por el macro a esto:
(reduce + 0 (map :total (filter :activa? ordenes)))

;; El macro solo reordena el código en tiempo de compilación

```

Ejemplo de macro propio:

```
(defmacro cuando-ok [resultado & cuerpo]
  `(when (:ok ~resultado)
    ~@cuerpo))

;; Uso:
(cuando-ok (dividir 10 2)
  (println "Resultado:" (:value (dividir 10 2))))
```

“->> no es una función: es un macro. Por eso puede recibir formas sin evaluar y reorganizarlas. No podría ser una función porque los argumentos de una función se evalúan antes de la llamada.”

Por qué usarlos con cuidado: - Son potentes, pero oscurecen el flujo de datos si se abusa. - Difíciles de debuggear (el error aparece en el código expandido, no en el macro). - Regla práctica: si se puede resolver con una función, usar función. Los macros son para casos donde el orden de evaluación importa (DSLs, control de flujo nuevo).

Transición: “Pasemos al tema que más justifica el funcional en sistemas reales: la concurrencia.”

BLOQUE 3 — Concurrencia y metaprogramación (30 min)

[F-25] Concurrencia funcional: por qué

Tiempo: 3 min

Al mostrar el concepto > “Si hay un dominio donde la programación funcional entrega su mayor valor, es la concurrencia. El razón es directa: la mayoría de los bugs de concurrencia existen porque dos hilos acceden al mismo estado mutable al mismo tiempo. Si el estado no puede mutarse, esa clase de bug desaparece.”

Los tres problemas clásicos del estado compartido: 1. **Race condition:** dos threads leen-modifican-escriben el mismo valor sin coordinación. El resultado depende del timing de la CPU. 2. **Deadlock:** dos threads esperan un recurso que el otro tiene. El programa se congela. 3. **Inconsistencia de cache:** en sistemas multi-core, cada núcleo tiene su cache. Sin memoria ordenada, ven versiones distintas del mismo valor.

Cómo responde el funcional: > “Si los valores son inmutables, dos threads pueden leer el mismo objeto simultáneamente sin ningún problema: no hay escritura, no hay race condition. La sincronización costosa se necesita solo cuando hay que *actualizar* estado, y hay modelos específicos para eso.”

Los cuatro modelos de Clojure para estado concurrente: - `ref` + `dosync` — transacciones coordinadas entre múltiples valores (STM) - `atom` — un único valor con actualizaciones atómicas (visto brevemente en F-28) - `agent` — actualizaciones asíncronas fuera del thread principal - `core.async` — comunicación entre procesos ligeros mediante canales (CSP)

[F-26] `core.async`: canales en Clojure

Tiempo: 3 min

Al mostrar el ejemplo parcial > “`core.async` trae al JVM (y a ClojureScript) el modelo CSP: Communicating Sequential Processes. La idea central es que en lugar de compartir memoria, los procesos se comunican pasándose mensajes a través de *canales*.”

¿Qué es un canal? - Un canal es como una tubería segura para transmitir valores entre procesos. - Tiene un buffer opcional. Si el buffer está lleno, el productor espera. Si está vacío, el consumidor espera. Esa espera es asíncrona — no bloquea el thread.

Código de ejemplo:

```
(require '[clojure.core.async :refer [chan go >! <!]])

;; Crear canal con buffer de 10
(def canal (chan 10))

;; Productor: produce valores en un go block
(go
  (>! canal 42)    ; poner valor en el canal (asíncronico)
  (>! canal 99))

;; Consumidor: lee del canal
(go
  (println "Recibi:" (<! canal)) ; => 42
  (println "Recibi:" (<! canal))) ; => 99
```

Separación productor/consumidor: > “El productor no sabe quién va a leer el canal, ni cuándo. El consumidor no sabe quién produce ni desde dónde. Este desacoplamiento es clave para sistemas de eventos en tiempo real, donde la tasa de producción y consumo pueden ser distintas.”

[F-27] go blocks y comunicación

Tiempo: 3 min

Al mostrar el código > “Los go blocks son procesos ligeros que `core.async` ejecuta sobre un pool de threads. Dentro de un go, podemos escribir código que parece secuencial pero es asíncrono: cuando esperamos en una operación de canal, el thread se libera y procesa otro go.”

Código completo productor-consumidor:

```
(require '[clojure.core.async :refer [chan go >! <! close!]])

(defn pipeline-ordenes [ordenes]
  (let [entrada (chan 5)
        salida  (chan 5)]

    ;; Productor: manda órdenes al canal
    (go
      (doseq [orden ordenes]
        (>! entrada orden))
      (close! entrada))

    ;; Transformador: lee de entrada, escribe en salida
    (go
      (loop []
        (when-let [orden (<! entrada)]
          (when (:activa? orden)
            (>! salida (:total orden)))
          (recur))))

    salida)) ; devuelve el canal de salida
```

Puntos para desarrollar: 1. `>!` — poner en el canal (parking si está lleno). `<!` — tomar del canal (parking si está vacío). 2. `close!` — cierra el canal. Los consumidores reciben `nil` cuando el canal cerrado se agota. 3. `when-let [orden (<! entrada)]` — patrón estándar: si el canal está cerrado y vacío, `<!` devuelve `nil` y el `when-let` termina el loop. 4. Los go blocks no bloquean threads — son coroutines sobre un pool de threads (por defecto 8 en la JVM).

“La diferencia con threads Java: un thread bloqueado consume recursos. Un go bloqueado libera el thread. Podemos tener miles de go blocks activos con solo decenas de threads reales.”

[F-28] STM y transacciones

Tiempo: 3 min

Al mostrar el concepto > “El STM (Software Transactional Memory) de Clojure resuelve el problema de coordinar actualizaciones a múltiples valores al mismo tiempo, de forma atómica y sin deadlocks.”

El problema que resuelve:

```
;; Transferencia bancaria entre dos cuentas  
;; SIN STM: race condition clásico  
;; Si dos transfers ocurren simultáneamente, podemos perder dinero
```

Con STM — ref y dosync:

```
(def cuenta-a (ref 1000))  
(def cuenta-b (ref 500))  
  
(defn transferir [origen destino monto]  
  (dosync  
    (alter origen - monto)    ; restar del origen  
    (alter destino + monto))) ; agregar al destino  
  
;; Ambas operaciones son atómicas: las dos pasan o ninguna  
(transferir cuenta-a cuenta-b 200)  
;; cuenta-a => 800, cuenta-b => 700
```

Cómo funciona dosync: 1. Cada ref lleva una versión (timestamp). 2. dosync ejecuta el bloque en un contexto transaccional: lee los valores actuales. 3. Al final del bloque, verifica que nadie los modificó mientras tanto. 4. Si hubo conflicto, **reinicia** la transacción automáticamente. Sin locks manuales. 5. Si no hubo conflicto, aplica los cambios atómicamente.

“El STM transforma el modelo mental: en lugar de pensar en locks (que pueden causar deadlocks), pensamos en transacciones. Si el sistema detectó conflicto, reintenta. El programador nunca maneja eso explícitamente.”

[F-29] Agentes y estado asíncrono

Tiempo: 3 min

Al mostrar el concepto > “Un **agent** en Clojure es una referencia a un valor que se actualiza *asíncronamente*. Las acciones se envían al agente y se ejecutan en un thread separado, en cola, de a una por vez. El agente nunca bloquea al que envía la acción.”

Código de ejemplo:

```
(def contador (agent 0))

;; Enviar acciones (no bloqueante):
(send contador inc)      ; encola inc
(send contador inc)      ; encola otro inc
(send contador + 10)     ; encola + 10

;; En algún momento futuro, el agente procesa en orden:
;; 0 -> 1 -> 2 -> 12

;; Leer el valor actual (siempre consistente):
@contador ; => 12 (si ya se procesaron todas)
```

Diferencias entre los modelos: | Mecanismo | Uso óptimo | |—|—| | **atom** | Un solo valor, actualizaciones simples y frecuentes | | **ref** + STM | Múltiples valores coordinados en una transacción | | **agent** | Actualizaciones asíncronas a un valor, sin urgencia | | **core.async** | Comunicación de mensajes, pipelines de eventos |

Cuándo es la buena opción: > “Los **agents** son ideales para efectos secundarios asíncrono que no requieren respuesta inmediata: loguear eventos, actualizar un contador de métricas, notificar a un sistema externo. El thread principal no espera al agente.”

[F-30] Concurrency en TypeScript

Tiempo: 3 min

Al mostrar el concepto > “TypeScript corre en JavaScript, que es monohilo con un event loop. No hay concurrencia real en el sentido de múltiples threads ejecutando código de usuario simultáneamente. Pero hay *asíncronismo*: podemos hacer I/O (llamadas de red, disco) sin bloquear el hilo principal.”

El modelo conceptual de Promise:

```
// Una Promise es un contenedor de un valor que llegará en el futuro
// Puede estar en tres estados:
// - pending: aún no resolvió
```

```
// - fulfilled: resolvió con valor
// - rejected: falló con error

const promesa: Promise<number> = fetch('/api/datos')
  .then(res => res.json())
  .then(data => data.total);
```

Dónde se pierde la pureza: > “Una función que devuelve `Promise<number>` está declarando que hará un efecto (I/O). El valor futuro depende de factores externos: la red, el servidor, el tiempo. Eso ya no es una función pura — y eso está bien. Lo importante es *aislar* esa impureza y no mezclarla con la lógica pura.”

Comparación con Clojure: - En Clojure, los `go blocks` de `core.async` son la forma funcional de modelar lo mismo: valores que llegan en el futuro, sin bloquear threads. - La diferencia conceptual: `Promise` en JS resuelve *una* vez. Un canal de `core.async` puede emitir *muchos* valores a lo largo del tiempo.

[F-31] Promesas y `async-await`

Tiempo: 3 min

Al mostrar el código > “`async/await` es azúcar sintáctica sobre Promises. Nos permite escribir código asíncronico que se lee como código sincrónico secuencial. Es mucho más legible que los `.then()` anidados.”

Ejemplo completo:

```
// Sin async/await (callback hell con .then):
fetch('/api/ordenes')
  .then(res => res.json())
  .then(ordenes => ordenes.filter((o: any) => o.activa))
  .then(activas => activas.map((o: any) => o.total))
  .then(totales => totales.reduce((a: number, b: number) => a + b, 0))
  .then(console.log)
  .catch(console.error);

// Con async/await (secuencial y legible):
async function totalOrdenesActivas(): Promise<number> {
  const res      = await fetch('/api/ordenes');      // efecto: I/O de red
  const ordenes  = await res.json();                  // efecto: parse JSON
```

```

// Desde acá, lógica pura:
return ordenes
  .filter((o: any) => o.activa)
  .map((o: any) => o.total)
  .reduce((a: number, b: number) => a + b, 0);
}

```

Puntos para desarrollar: 1. El `await` suspende la función actual hasta que la Promise resuelve, pero *no bloquea el thread*. El event loop procesa otras tareas mientras espera. 2. Los efectos (I/O) están al inicio. La lógica pura (`filter`, `map`, `reduce`) está al final. Esta separación es la mejor práctica. 3. Los errores se manejan con `try/catch` dentro de la función `async`, o con `.catch()` al llamarla.

“Fíjenle que la lógica pura del final es testeable sin red. Podemos probar `filter`, `map` y `reduce` con datos locales. Solo el inicio — el `fetch` — necesita un mock en tests. Eso es separación de efectos.”

[F-32] Efectos puros vs I/O

Tiempo: 3 min

Al mostrar la comparación > “Último concepto de este bloque: la diferencia entre código puro y código con efectos, y la disciplina de separarlos.”

Definiciones:

```

// PURO: mismo input, mismo output. Sin I/O, sin estado global.
const calcularDescuento = (precio: number, pct: number): number =>
  precio * (1 - pct / 100);
// Test trivial: calcularDescuento(100, 20) === 80. Siempre.

// CON EFECTO: depende de o modifica el mundo exterior.
const guardarOrden = async (orden: Orden): Promise<void> => {
  await db.insert('ordenes', orden); // efecto: escritura en base de datos
};
// Test requiere mock de la base de datos.

```

El principio de separación: > “La mejor arquitectura funcional consiste en un núcleo de lógica pura rodeado por una capa delgada de efectos. Los efectos llaman al núcleo puro; el núcleo puro nunca toca el mundo exterior.”

Diagrama para pizarra:

```
[I/O entrada] → [Lógica pura] → [I/O salida]
  fetch(url)      filter/map/reduce  guardarOrden()
  readFile()      validar/calcular    sendEmail()
```

En Haskell y lenguajes puramente funcionales: - El sistema de tipos *impone* la separación: las funciones con efectos solo pueden llamarse desde contextos IO. En TS/Clojure es convención, no obligación.

Cuándo *no* separar: > “A veces el efecto está tan entretelado con la lógica que separarlos agrega más complejidad que valor. Ser pragmático: aplicar el principio donde el beneficio en testabilidad y legibilidad es real.”

[F-33] Canal vs promesa

Tiempo: 3 min

Al mostrar la tabla > “Comparamos el modelo de canales de `core.async` con el modelo de Promises de TypeScript para entender cuándo elegir cada uno.”

Dimensión	<code>core.async</code> canal	Promise en TypeScript
Cardinalidad	Emite MÚltiples valores a lo largo del tiempo	Resuelve UNA única vez
Dirección	Bidireccional (se puede escribir y leer)	Unidireccional (productor → consumidor)
Backpressure	Sí: el productor espera si el buffer está lleno	No nativo
Composición	Canales se pueden conectar en pipelines	<code>.then()</code> / <code>async-await</code>
Cancelación	<code>close!</code> cancela el canal	<code>AbortController</code> (más verbose)
Caso de uso ideal	Flujos de eventos continuos (teclado, sensores, websockets)	Petición-respuesta única (API REST, archivo)

Equivalente de canal en TS: AsyncGenerator:

```
async function* eventosTeclado(): AsyncGenerator<string> {
  // emite múltiples eventos
  for await (const evento of stream) {
    yield evento.key;
```

```

    }
  }
  // Uso:
  for await (const key of eventosTeclado()) {
    procesar(key);
  }

```

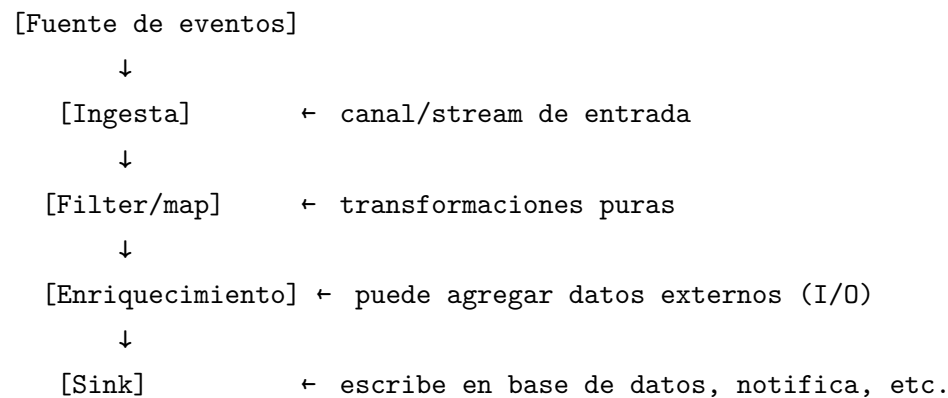
Pregunta: “¿Cuál modelo es más natural para un chat en tiempo real? ¿Y para buscar un usuario por ID? (Respuesta esperada: canal/AsyncGenerator para el chat; Promise para la búsqueda.)”

[F-34] Diseño de flujo continuo

Tiempo: 3 min

Al mostrar el diagrama > “Cerramos el bloque con un patrón arquitectural: el pipeline de procesamiento continuo. Es el patrón que usan sistemas como Kafka, RxJS y `core.async` para procesar flujos de datos en tiempo real.”

Descripción del flujo:



Ejemplo en Clojure con `core.async`:

```

(defn iniciar-pipeline [eventos-ch]
  (let [filtrados (chan 100)
        mapeados  (chan 100)]

    ;; Etapa 1: filtrar
    (go-loop []
      (when-let [evt (<! eventos-ch)]
        (when (:relevante? evt)
          (>! filtrados evt))

```

```

(recur)))

;; Etapa 2: transformar
(go-loop []
  (when-let [evt (<! filtrados)]
    (>! mapeados (enriquecer evt))
    (recur)))

mapeados)) ; canal final del pipeline

```

Relevancia en producción: > “Este patrón es exactamente lo que usa Kafka Streams, Apache Flink y cualquier sistema de procesamiento de eventos. La abstracción del canal desacopla cada etapa: podemos escalar, reemplazar o monitorear cada parte independientemente.”

BLOQUE 4 — Práctica guiada y reflexión (20 min)

[F-35] Taller comparativo

Tiempo: 8 min

Al presentar el desafío > “Vamos a poner en práctica todo lo que vimos. El desafío es claro: procesar un dataset de órdenes de compra usando los patrones que aprendimos. Mitad del grupo en TypeScript, mitad en Clojure. 8 minutos.”

Contexto del problema:

```

// Dataset compartido (adaptar a Clojure en el otro grupo):
const ordenes = [
  { id: 1, cliente: "Ana", total: 250, categoria: "elect", activa: true },
  { id: 2, cliente: "Boris", total: 80, categoria: "ropa", activa: false },
  { id: 3, cliente: "Carla", total: 420, categoria: "elect", activa: true },
  { id: 4, cliente: "Diana", total: 30, categoria: "ropa", activa: true },
  { id: 5, cliente: "Edwin", total: 175, categoria: "elect", activa: true },
];

```

Objetivo: 1. Filtrar solo las órdenes activas. 2. De esas, tomar solo las de categoría "elect". 3. Si el total supera 200, devolver `Result.ok(total)`, si no, `Result.err("monto insuficiente")`. 4. Sumar solo los ok.

Instrucción práctica: - Repartir roles: la mitad trabaja en TS (pueden usar papel o IDE), la otra en Clojure (papel). - Usar `Result/Either` en TS y mapas `{:ok}` en Clojure. - No es necesario

que corra: lo importante es el diseño de tipos y el pipeline.

[F-36] Guion TS del taller

Tiempo: 3 min

Al mostrar el código base > “Para el equipo TypeScript, les doy la estructura de tipos. Los pipelines son suyos.”

```
type Result<T, E = string> =
  | { ok: true; value: T }
  | { ok: false; error: E };

const ok = <T>(v: T): Result<T, never> => ({ ok: true, value: v });
const err = <E>(e: E): Result<never, E> => ({ ok: false, error: e });

type Orden = {
  id: number; cliente: string; total: number;
  categoria: string; activa: boolean;
};

// Función a completar:
const clasificarOrden = (o: Orden): Result<number, string> => {
  // TODO: si total > 200, ok(total); si no, err("monto insuficiente")
};

// Pipeline a construir:
const totalElectActivos = (ordenes: Orden[]): number =>
  ordenes
    // TODO: filter activa
    // TODO: filter categoria === "elect"
    // TODO: map a Result con clasificarOrden
    // TODO: filter los ok
    // TODO: sumar los valores
    .reduce((acc, _) => acc, 0);
```

Pista si están trabados: “¿Cómo filtran los `Result.ok` del array? Recuerden que el discriminante es la propiedad `.ok`.”

[F-37] Guion Clojure del taller

Tiempo: 3 min

Al mostrar el código base > “Para el equipo Clojure, les doy la estructura de datos. El pipeline y la lógica de validación son suyos.”

```
(def ordenes
  [{:id 1 :cliente "Ana"   :total 250 :categoria "elect" :activa? true}
   {:id 2 :cliente "Boris" :total 80  :categoria "ropa"  :activa? false}
   {:id 3 :cliente "Carla" :total 420 :categoria "elect" :activa? true}
   {:id 4 :cliente "Diana" :total 30  :categoria "ropa"  :activa? true}
   {:id 5 :cliente "Edwin" :total 175 :categoria "elect" :activa? true}])

;; Función a completar:
(defn clasificar-orden [orden]
  ;; TODO: si :total > 200, {:ok true :value (:total orden)}
  ;;      si no,           {:ok false :error "monto insuficiente"}
  )

;; Pipeline a construir (dos opciones):

;; Opción 1: con ->>:
(defn total-elect-activos [ordenes]
  (->> ordenes
    ;; TODO: filter :activa?
    ;; TODO: filter #(= (:categoria %) "elect")
    ;; TODO: map clasificar-orden
    ;; TODO: filter :ok
    ;; TODO: map :value
    (reduce + 0)))

;; Opción 2 (avanzada): con transduce
```

Pista si están trabados: “¿Cómo filtrar los mapas con :ok true? Recuerden que :ok como keyword se comporta como predicado.”

[F-38] Comparar soluciones

Tiempo: 2 min

Al guiar la puesta en común > “Vamos a poner en el pizarrón las dos soluciones lado a lado. Me interesa que vean las similitudes y las diferencias.”

Preguntas para guiar la comparación: 1. “¿dónde definieron los tipos o el contrato del dato en cada lenguaje? En TS: interfaz/type. En Clojure: implícitamente en el mapa. ¿Cuál les parece más claro?” 2. “¿Cómo manejaron el **Result**? En TS el tipo les ayudó a no olvidar el caso de error. En Clojure, ¿qué pasa si alguien se olvida de filtrar los `{:ok false}`?” 3. “¿Alguien llegó a usar **transduce** en Clojure? ¿Qué cambió?”

Conceptos que deben aparecer en la comparación: - Ambos usan **filter**, **map**, **reduce** como primitivas. - El tipo algebraico en TS agrega garantías en compilación. En Clojure, la garantía es convención + tests. - El pipeline de Clojure es generalmente más conciso. El de TS es más explícito.

Punt clave para reforzar: > “Los mismos *principios* — inmutabilidad, funciones puras, manejo explícito de errores, composición — aparecen en ambos lenguajes. La sintaxis es diferente, el pensamiento es el mismo.”

[F-39] Buenas preguntas para el cierre

Tiempo: 1 min

Sugerir preguntas - ¿Qué abstraemos con un **Result**? - ¿En qué caso elegimos `core.async`? - ¿Cuál es la diferencia clave entre pipeline y transducer?

[F-40] Evaluación pedagógica rápida

Tiempo: 1 min

Revisar indicadores - Comprensión de patrones funcionales - Manejo de errores explícito - Concurrencia funcional definida

[F-41] Resumen final

Tiempo: 1 min

Reforzar lo esencial - Menos estado mutable, más composición - **Result** hace el flujo explícito - Clojure y TypeScript comparten los mismos principios

[F-42] Próxima clase y TP

Tiempo: 1 min

Concluir con la agenda siguiente - Tema siguiente: Mónadas en TypeScript - TP: implementar una API funcional y justificar la elección de efectos

Materiales y recursos en clase

- Código de ejemplo TypeScript con `readonly`, `Result`, `compose`
 - Código de ejemplo Clojure con `->>`, `transduce`, `core.async`
 - Pizarra: diagrama comparativo TS Clojure
 - Ejercicio en parejas con dataset de órdenes
-

Trazabilidad a filminas

- F-01 → Portada del tema
- F-02 → ¿Por qué hablar de funcional?
- F-03 → Imperativo vs funcional
- F-04 → Funciones puras
- F-05 → Inmutabilidad
- F-06 → Pipeline TS
- F-07 → `filter` + `map`
- F-08 → `reduce`
- F-09 → Composición TS
- F-10 → Colecciones inmutables TS
- F-11 → Secuencias perezosas Clojure
- F-12 → Pipeline Clojure
- F-13 → Colecciones persistentes Clojure
- F-14 → Algebraic data types TS
- F-15 → `Result` vs excepción
- F-16 → Ejemplo `Result` TS
- F-17 → `Option` / `Maybe`
- F-18 → Manejo de errores Clojure
- F-19 → ¿Qué es un transducer?
- F-20 → Ejemplo de `transduce`
- F-21 → Transducers vs pipeline convencional
- F-22 → API funcional genérica TS

- F-23 → Funciones de orden superior
- F-24 → Metaprogramación en Clojure
- F-25 → Concurrency funcional: por qué
- F-26 → `core.async` canales
- F-27 → `go` blocks
- F-28 → STM y transacciones
- F-29 → Agentes Clojure
- F-30 → Concurrency TS
- F-31 → Promesas y `async-await`
- F-32 → Efectos puros vs I/O
- F-33 → Canal vs promesa
- F-34 → Diseño de flujo continuo
- F-35 → Taller comparativo
- F-36 → Guion TS del taller
- F-37 → Guion Clojure del taller
- F-38 → Comparar soluciones
- F-39 → Buenas preguntas para el cierre
- F-40 → Evaluación pedagógica rápida
- F-41 → Resumen final
- F-42 → Próxima clase y TP